

Frontend + Docker Deployment Guide for Your RAG Chatbot

You already have:

- Working RAG backend
- Ollama + LLaMA
- ChromaDB
- PDF retrieval
- Linux server

Now we will build:

1. Frontend UI
2. API backend
3. Docker deployment
4. Cross-platform setup

Goal:

Your chatbot should run properly on:

- Linux
- Windows
- Another computer
- Another server

without breaking.

FINAL ARCHITECTURE

```
graph TD; Frontend["Frontend (Gradio)"] --> Backend["FastAPI Backend"]; Backend --> Pipeline["RAG Pipeline"]; Pipeline --> ChromaDB["ChromaDB"]; ChromaDB --> Ollama["Ollama + LLaMA"];
```

STEP 1 — INSTALL EXTRA PACKAGES

Inside your virtual environment:

```
pip install fastapi uvicorn gradio
```

STEP 2 — CREATE BACKEND API

Create file:

```
nano backend.py
```

Paste this COMPLETE code:

```
import os

from fastapi import FastAPI
from pydantic import BaseModel

from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.llms import Ollama
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory

# =====
# SETTINGS
# =====

DOCUMENTS_PATH = "documents"
DB_PATH = "chroma_db"

# =====
# LOAD DOCUMENTS
# =====

all_docs = []

print("Loading documents...")

for file in os.listdir(DOCUMENTS_PATH):

    if file.endswith(".pdf"):
```

```

        file_path = os.path.join(DOCUMENTS_PATH, file)

        loader = PyPDFLoader(file_path)
        docs = loader.load()

        all_docs.extend(docs)

print(f"Loaded pages: {len(all_docs)}")

# =====
# SPLIT DOCUMENTS
# =====

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1200,
    chunk_overlap=200
)

texts = text_splitter.split_documents(all_docs)

print(f"Chunks created: {len(texts)}")

# =====
# EMBEDDINGS
# =====

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# =====
# VECTOR DATABASE
# =====

if os.path.exists(DB_PATH):

    vector_db = Chroma(
        persist_directory=DB_PATH,
        embedding_function=embedding_model
    )

else:

    vector_db = Chroma.from_documents(
        documents=texts,
        embedding=embedding_model,
        persist_directory=DB_PATH
    )

    vector_db.persist()

```

```

retriever = vector_db.as_retriever(
    search_kwargs={"k": 5}
)

# =====
# LLM
# =====

llm = Ollama(model="llama3.2")

# =====
# MEMORY
# =====

memory = ConversationBufferMemory(
    memory_key="chat_history",
    return_messages=True,
    output_key="answer"
)

# =====
# RAG CHAIN
# =====

qa_chain = ConversationalRetrievalChain.from_llm(
    llm=llm,
    retriever=retriever,
    memory=memory,
    return_source_documents=True
)

# =====
# FASTAPI
# =====

app = FastAPI()

class ChatRequest(BaseModel):
    question: str

@app.post("/chat")
def chat(request: ChatRequest):

    result = qa_chain.invoke({
        "question": request.question
    })

    answer = result["answer"]

```

```
sources = []

for doc in result["source_documents"]:

    source = doc.metadata.get("source", "Unknown")
    page = doc.metadata.get("page", "?")

    sources.append({
        "source": source,
        "page": page
    })

return {
    "answer": answer,
    "sources": sources
}
```

STEP 3 — RUN BACKEND

Run:

```
uvicorn backend:app --host 0.0.0.0 --port 8000
```

If successful:

```
Application startup complete
```

STEP 4 — TEST BACKEND

Open browser:

```
http://YOUR_SERVER_IP:8000/docs
```

FastAPI automatically creates Swagger UI.

You can test the API there.

STEP 5 — CREATE FRONTEND

Create file:

```
nano frontend.py
```

Paste this code:

```
import requests
import gradio as gr

API_URL = "http://127.0.0.1:8000/chat"

def chatbot(message, history):

    try:

        response = requests.post(
            API_URL,
            json={"question": message}
        )

        data = response.json()

        answer = data["answer"]

        sources = data["sources"]

        source_text = "\n\nSources:\n"

        for s in sources:
            source_text += f"- {s['source']} | Page {s['page']}\n"

        return answer + source_text

    except Exception as e:
        return f"Error: {e}"

interface = gr.ChatInterface(
    fn=chatbot,
    title="RAG AI Assistant",
    description="Ask questions from uploaded PDFs",
    theme="soft"
)

interface.launch()
```

```
server_name="0.0.0.0",  
server_port=7860  
)
```

STEP 6 — RUN FRONTEND

Open SECOND terminal.

Activate venv:

```
cd ~/rag-chatbot  
source venv/bin/activate
```

Run:

```
python frontend.py
```

STEP 7 — ACCESS CHATBOT

Open browser:

```
http://YOUR_SERVER_IP:7860
```

Now your chatbot has:

- beautiful UI
- backend API
- RAG system
- source citations
- conversational retrieval

STEP 8 — CREATE DOCKER FILE

Docker makes your chatbot portable.

Create file:

```
nano Dockerfile
```

Paste:

```
FROM python:3.11

WORKDIR /app

COPY . .

RUN pip install --upgrade pip

RUN pip install fastapi uvicorn gradio requests
RUN pip install langchain==0.1.20
RUN pip install langchain-community==0.0.38
RUN pip install langchain-text-splitters==0.0.2
RUN pip install chromadb sentence-transformers pypdf ollama

EXPOSE 7860
EXPOSE 8000

CMD ["python", "frontend.py"]
```

STEP 9 — CREATE REQUIREMENTS FILE

Run:

```
pip freeze > requirements.txt
```

STEP 10 — BUILD DOCKER IMAGE

Run:

```
docker build -t rag-chatbot .
```

STEP 11 — RUN DOCKER CONTAINER

```
docker run -p 7860:7860 -p 8000:8000 rag-chatbot
```

IMPORTANT NOTE ABOUT OLLAMA

Ollama must also be installed on the deployment machine.

And the model must exist:

```
ollama pull llama3.2
```

HOW TO RUN ON ANOTHER COMPUTER

On another machine:

1. Install Docker
2. Install Ollama
3. Pull llama model
4. Copy project
5. Run container

That is why Docker is useful.

FUTURE IMPROVEMENTS

You can later add:

- file upload button
- scanned PDF OCR
- React frontend
- authentication
- streaming responses
- multilingual support
- cloud deployment

WHAT TO SAY IN VIVA

"I developed a scalable RAG architecture using Ollama-hosted LLaMA, LangChain orchestration, Chroma vector retrieval, FastAPI backend services, and a Gradio frontend interface. Dockerization was used to improve portability and cross-platform deployment compatibility."